ai-rossby

Deep-learning weather & climate emulators on PhysicsNeMo

Group onboarding

Why

- Train **fast surrogate models** for weather & climate (PLASIM, ERA5, E3SM, AMIP)
- One codebase, many model families — shared data pipeline, training loop, evaluation
- Built on **NVIDIA PhysicsNeMo** (we maintain a fork)

Lineage

- **Fork of NVIDIA PhysicsNeMo** (Apache-2.0) — the framework
- Deterministic models derived from **PanguWeather v2.0** (→ makani / FourCastNet → Pangu-Weather)
- Diffusion models derived from the **amip** codebase *(experimental)*
- See `NOTICE` for provenance & licensing

Supported models

Family	What	Status
PanguPlasim / Legacy	Pangu-style transformer	✅
SfnoPlasim / SFNO-E3SM	Spherical FNO	✅
AMIP diffusion (SI/EDM/...)	latent diffusion	⚠️ experimental

Start with the deterministic Pangu/SFNO recipes.

The data: Zarr stores

- Recipe reads **Zarr** — one store per split + small normalization/climatology stores
- Sources: **E3SM, PLASIM, ERA5, AMIP** (raw per-timestep HDF5 → Zarr via `tools/data/`)
- On **Delta** the converted stores already exist (group-readable)

Data: pointing at it

```
# On Delta: works with no setup (configs fall back to the shared location)
# Elsewhere: set one env var
export AI_ROSSBY_DATA=/my/physicsnemo-zarr
```

Details + the converted-store table: `examples/weather/ai_rossby/DATA.md`

Repo tour

- `examples/weather/ai_rossby/` — the recipe (**train / inference / validate**)
- `examples/weather/ai_rossby/conf/` — Hydra configs (model / dataset / training / loss / validation)
- `tools/` — data conversion + checkpoint translation
- `hpc/` — per-cluster install & run docs
- `docs/dev/` — internal design/plan history

Configs are composable (Hydra)

Pick one YAML per group, override any leaf on the CLI:

```
python train.py \
  model=sfno_e3sm dataset=e3sm training=sfno_plasim \
  loss=raw_l2 validation=off \
  training.max_epochs=100 dataset.batch_size=8
```

Set up the environment

- Portable recipe: `hpc/install.md`
- Per-cluster: `hpc/delta.md`, `hpc/deltaai.md`, `hpc/derecho.md`, …
- `uv` + the system PyTorch module; a per-cluster venv

Train — single GPU

```
cd examples/weather/ai_rossby
python train.py \
  model=sfno_e3sm dataset=e3sm training=sfno_plasim \
  run_name=my_first_run
```

Checkpoints → `outputs/my_first_run/checkpoints/`

Train — multi-GPU

```
torchrun --standalone --nproc-per-node=4 train.py \
  model=sfno_e3sm dataset=e3sm training=sfno_plasim \
  run_name=my_first_ddp
```

⚠️ SFNO+DDP needs **torch<2.11** (handled by the pinned env)

Monitor

- **wandb** (default, offline — local `./wandb/`, no login)
- `train/*` loss components · `valid/*` val_loss / RMSE / ACC
- `wandb sync ...` to upload; `wandb.mode=online` to stream

Evaluate

```
python inference.py model=sfno_e3sm dataset=e3sm \
  +inference.checkpoint_dir=... +inference.output_path=preds.nc

python validate_cli.py dataset=e3sm \
  +validation_cli.predictions=preds.nc \
  +validation_cli.reference_zarr=$AI_ROSSBY_DATA/e3sm/2045.zarr
```

Clusters & scheduling

- **Delta / DeltaAI** (NCSA), **Derecho** (NCAR), **Stampede3** (TACC), **Midway3** & **DSI** (UChicago)
- SLURM/PBS smoke-test skills per cluster + example sbatch in `hpc/scripts/`
- Use **your** account/allocation (edit the sbatch headers)

Where to get help

- **Start:** top-level `README.md` → recipe `README.md`
- Data: `DATA.md` · Porting: `PANGUWEATHER_MIGRATION.md`
- Tools: `tools/README.md` · History: `docs/dev/`
- **Ask:** Alexander Wikner — awikner@uchicago.edu

First task

1. Set up your cluster env (`hpc/<cluster>.md`)
2. Run the single-GPU SFNO-E3SM train command
3. Watch `train/loss` fall
4. Roll out + score with `inference.py` → `validate_cli.py`

Welcome aboard.